Лабораторна робота №2

Забезпечення якості програмних проєктів через багаторівневе тестування

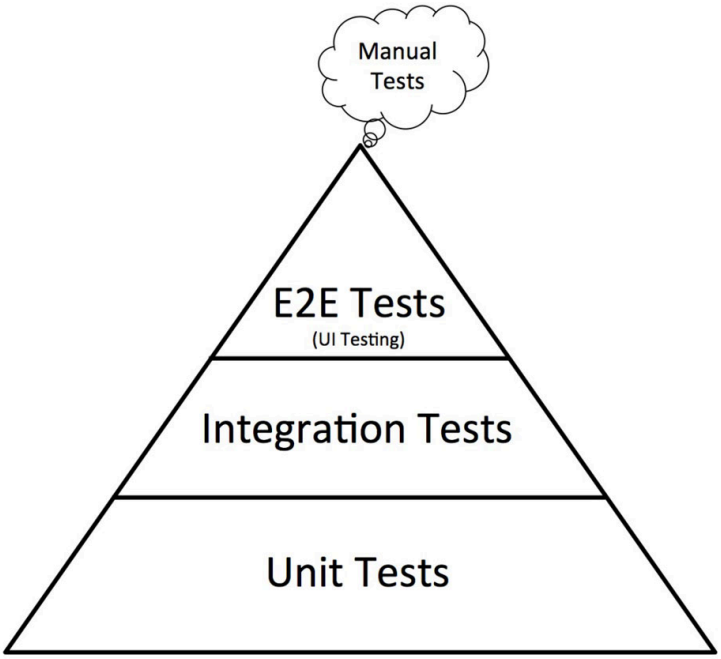
Мета роботи: Опанувати методології та інструменти автоматизованого тестування. Навчитися розробляти Unit-тести для бізнес-логіки, інтеграційні тести та наскрізні (E2E) тести для інтерфейсу користувача. Зрозуміти роль метрик покриття коду (Code Coverage) у прийнятті рішень про готовність продукту до релізу.

1. Концептуальна модель піраміди тестування

Піраміда тестування — це стратегічна модель, яка групує автоматизовані тести за їхньою деталізацією та вартістю виконання. Вона допомагає збалансувати зусилля команди для досягнення максимальної стабільності продукту.

Піраміда тестування

1. Unit Tests (Модульні тести):
    - Основа піраміди. Перевіряють найменші ізольовані частини коду: функції, методи або класи.
    - Вони найшвидші у виконанні та найдешевші в розробці. Їх має бути найбільше (70–80% від усієї кількості тестів).
2. Integration Tests (Інтеграційні тести):
    - Перевіряють взаємодію між декількома модулями або компонентами системи (наприклад, чи правильно фронтенд отримує дані від API).
    - Допомагають виявити помилки в інтерфейсах та протоколах обміну даними.
3. End-to-End Tests (E2E / Наскрізні тести):
    - Верхівка піраміди. Емулюють реальну поведінку користувача в браузері (наприклад: "зайти на сайт" -> "натиснути кнопку" -> "перевірити результат").
    - Найповільніші та найважчі в підтримці, але дають найбільшу впевненість у працездатності всього продукту.



Поняття Code Coverage (Покриття коду)

Це кількісна метрика, яка показує, який відсоток сирцевого коду був виконаний під час запуску тестів.

- **Line Coverage:** Відсоток рядків коду, через які "пройшли" тести.
- **Function Coverage:** Скільки функцій або методів було викликано.
- **Branch Coverage:** Чи були перевірені всі логічні розгалуження (всі умови if/else).

LCOV - code coverage report

			Hit	Total	Coverage
Current view:	top level				
Test:	lcov.info	**Lines:**	1617	2715	59.6 %
Date:	2022-04-03 10:59:38	**Functions:**	0	0	-

Directory	Line Coverage			Functions	
src		100.0 %	10 / 10	-	0 / 0
src/common_widgets		68.8 %	132 / 192	-	0 / 0
src/exceptions		17.9 %	10 / 56	-	0 / 0
src/features/address/application		100.0 %	9 / 9	-	0 / 0
src/features/address/data		33.3 %	9 / 27	-	0 / 0
src/features/address/domain		2.5 %	1 / 40	-	0 / 0
src/features/address/presentation		100.0 %	66 / 66	-	0 / 0
src/features/authentication/application		92.3 %	24 / 26	-	0 / 0
src/features/authentication/data		29.2 %	19 / 65	-	0 / 0
src/features/authentication/domain		7.7 %	1 / 13	-	0 / 0
src/features/authentication/presentation/account		92.0 %	46 / 50	-	0 / 0
src/features/authentication/presentation/admin		6.2 %	1 / 16	-	0 / 0
src/features/authentication/presentation/sign_in		64.5 %	129 / 200	-	0 / 0
src/features/cart/application		97.4 %	76 / 78	-	0 / 0
src/features/cart/data/local_cart		34.6 %	9 / 26	-	0 / 0
src/features/cart/data/remote_cart		41.4 %	12 / 29	-	0 / 0
src/features/cart/domain		51.4 %	37 / 72	-	0 / 0
src/features/cart/presentation/add_to_cart		94.0 %	63 / 67	-	0 / 0
src/features/cart/presentation/cart_total		100.0 %	13 / 13	-	0 / 0
src/features/cart/presentation/shopping_cart		89.3 %	100 / 112	-	0 / 0

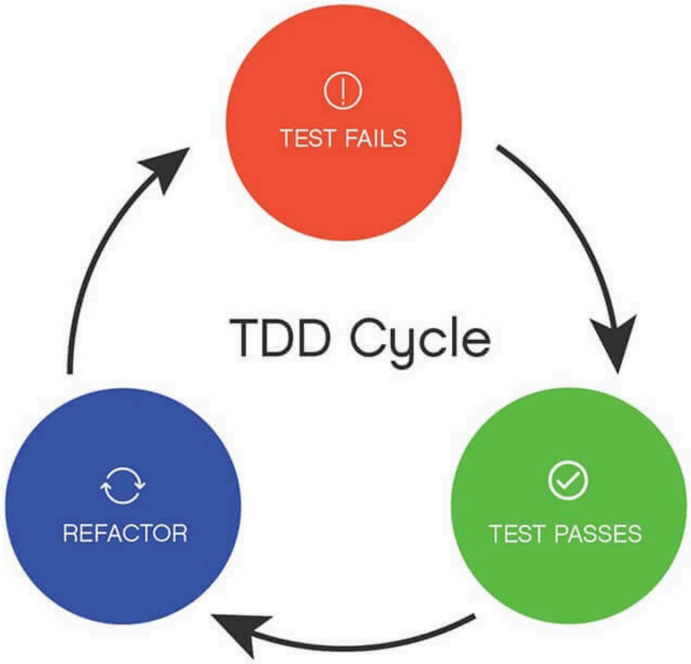
Важливо: Високе покриття (наприклад, 90%+) не гарантує відсутності багів, але воно вказує на те, що більшість логіки була активована і перевірена.

Методологія TDD (Test-Driven Development)

Розробка через тестування — це підхід, де тести пишуться перед написанням самого коду функціоналу.

Цикл розробки TDD (Red-Green-Refactor):

- **Red (Червоний):** Написати тест, який падає (бо функціонал ще не реалізовано).
- **Green (Зелений):** Написати мінімально необхідний код, щоб тест пройшов успішно.
- **Refactor (Рефакторинг):** Покращити структуру коду без зміни його поведінки.

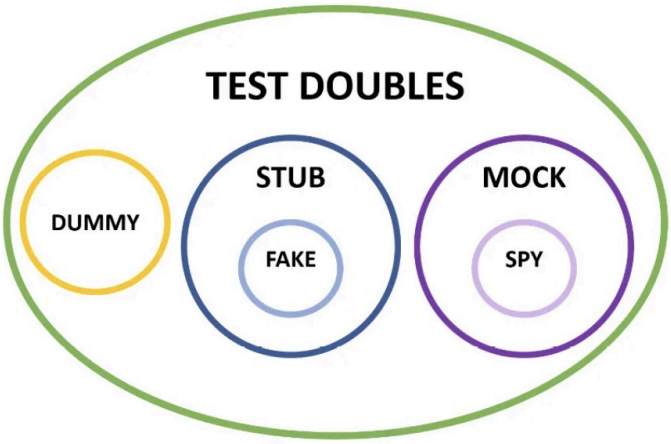


Ізоляція тестів: Mocks та Stubs

Для того, щоб Unit-тести були швидкими та стабільними, вони не повинні залежати від зовнішніх факторів (мережі, бази даних). Для цього використовують об'єкти-замінники:

- **Dummy (Фіктивний об'єкт / Муляж):** Це об'єкти, які передаються в систему виключно для задоволення вимог компілятора або підпису методу. Вони ніколи не використовуються під час виконання тесту, фактично виконуючи роль інертного заповнювача для обов'язкових параметрів, які не впливають на логіку конкретного сценарію.
- **Stub (Заглушка):** Надає жорстко визначені (жорстко закодовані) відповіді на виклики, які здійснює система під час тестування. Заглушки використовуються для того, щоб спрямувати хід виконання програми певним шляхом або надати коду необхідні вхідні дані, не реалізуючи при цьому жодної реальної логіки.

- **Fake (Фейк):** Це спрощена, але функціональна версія реального об'єкта, яка підходить для тестування, але не придатна для продакшену. Найчастішим прикладом є використання бази даних у пам’яті (In-memory DB), яка працює швидко і передбачувано, але не зберігає дані після завершення процесу.
- **Mock (Мок):** Об'єкт, який фокусується на перевірці взаємодії та поведінки, а не на стані даних. Він заздалегідь програмується на очікування конкретних викликів із певними аргументами, і якщо код не здійснює цих викликів у правильному порядку або з правильними параметрами, тест автоматично вважається проваленим.
- **Spy (Шпигун):** Своєрідна обгортка над реальним об'єктом, яка приховано записує інформацію про всі виклики методів (кількість, параметри, час). На відміну від мока, шпигун зазвичай дозволяє об'єкту виконувати його реальну логіку, надаючи розробнику можливість перевірити побічні ефекти вже після виконання основного сценарію.



Тестування "чорної скриньки" проти "білої скриньки"

Ці підходи відрізняються ступенем доступу до внутрішньої структури програми:

- **Тестування "чорної скриньки" (Black Box):** Тестувальник не знає внутрішньої будови коду. Перевіряється лише відповідність результату вхідним даним згідно зі специфікацією.
- **Тестування "білої скриньки" (White Box):** Тестувальник має доступ до коду і перевіряє внутрішні алгоритми, шляхи виконання, цикли та обробку винятків.

2. Індивідуальне завдання

Крок 1: Налаштування тестового оточення

Встановлення необхідних бібліотек у проєкт, створений у Лабі 1.

Налаштування скриптів запуску тестів у package.json або аналогічних файлах конфігурації.

```
{} package.json M ×
{} package.json > {} scripts
  "version": "0.0.2",
  "type": "module",
  "description": "Student Deadline Manager - MVP",
  ▷Debug
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "test:unit": "vitest",
    "test:unit:ui": "vitest --ui",
    "test:unit:coverage": "vitest --coverage",
    "test:e2e": "playwright test",
    "test:e2e:ui": "playwright test --ui",
    "test": "npm run test:unit && npm run test:e2e"
  },
  "dependencies": {
    "vue": "^3.4.21"
  },
  "devDependencies": {
    "@playwright/test": "^1.41.1",
    "@vitejs/plugin-vue": "^5.0.4",
    "@vitest/coverage-v8": "^1.6.1",
    "@vitest/ui": "^1.2.2",
    "@vue/test-utils": "^2.4.5",
    "autoprefixer": "^10.4.18",
    "jsdom": "^23.2.0",
    "postcss": "^8.4.35",
    "tailwindcss": "^3.4.1",
    "vite": "^5.1.4",
    "vitest": "^1.2.2"
  }
}
```

Крок 2: Розробка модульних тестів (Unit Tests)

Написання мінімум 5-7 тестів для ключових функцій бізнес-логіки вашого MVP.

Використання Assertions (перевірок) для валідації очікуваних результатів.

Практика з Mock-об’єктами для ізоляції логіки.

```
krostyslav@home:~/users/ros/dev/UniDone$
krostyslav@home:~/users/ros/dev/UniDone$ npm run test:unit

> unidone@0.0.2 test:unit
> vitest


 DEV  v1.6.1 /home/krostyslav/users/ros/dev/UniDone

 ✓ src/composables/__tests__/useTasks.test.js (13)
 ✓ src/App.test.js (3)

 Test Files  2 passed (2)
      Tests  16 passed (16)
   Start at  19:26:52
   Duration  928ms (transform 113ms, setup 16ms, collect 223ms, tests 46ms, environment 798ms, pr
epare 364ms)


 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

Крок 3: Аналіз покриття коду

Запуск інструменту генерації звіту про покриття (Coverage Report).

Аналіз ділянок коду, які не були охоплені тестами, та обґрунтування необхідності (або відсутності потреби) їхнього тестування.

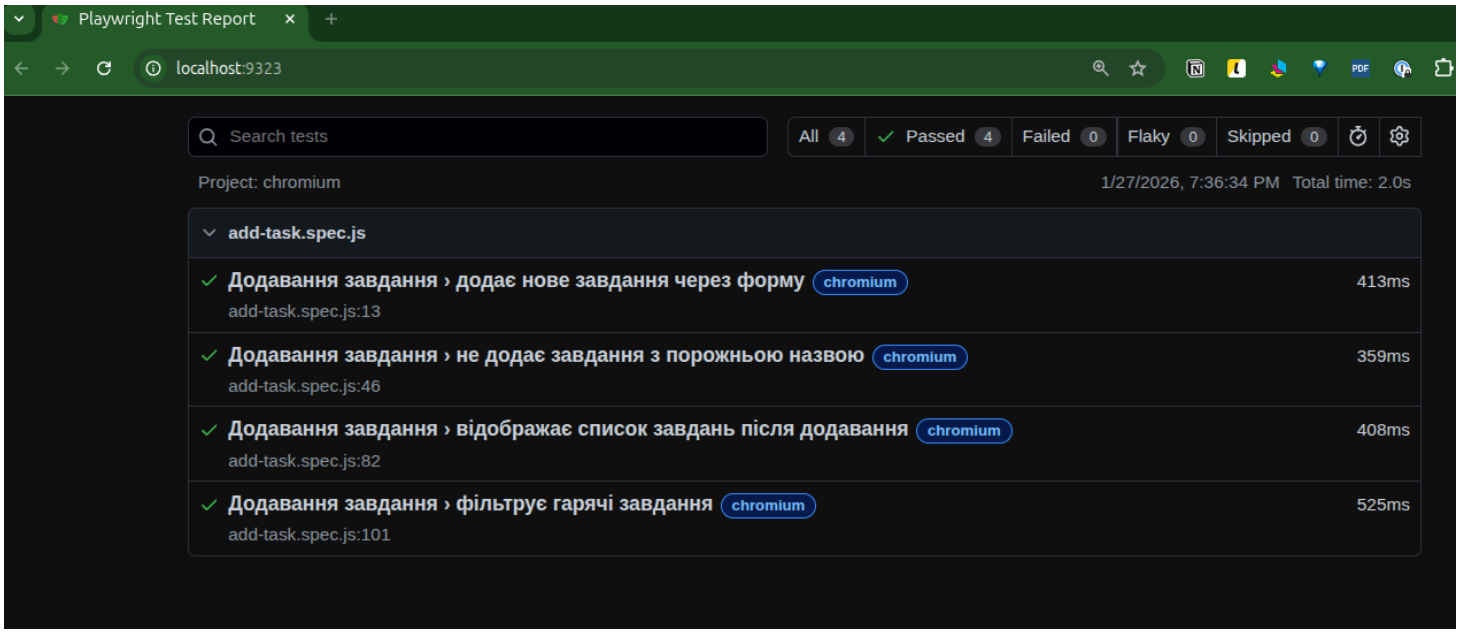
```
krostyslav@home:~/users/ros/dev/UniDone$ npm run test:unit:coverage

 > unidone@0.0.2 test:unit:coverage
 > vitest --coverage


 DEV  v1.6.1 /home/krostyslav/users/ros/dev/UniDone
      Coverage enabled with v8

  ✓ src/composables/__tests__/useTasks.test.js (13)
  ✓ src/App.test.js (3)

  Test Files  2 passed (2)
       Tests  16 passed (16)
    Start at  19:34:41
    Duration  800ms (transform 132ms, setup 26ms, collect 227ms, tests 50ms, environment 659ms, pr
 epare 144ms)

  % Coverage report from v8
-----------------|---------|----------|---------|---------|-------------------
File             | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----------------|---------|----------|---------|---------|-------------------
All files        |    94.6 |    81.48 |   47.36 |    94.6 |
 src             |   88.42 |     62.5 |   16.66 |   88.42 |
  App.vue        |   93.33 |    71.42 |      20 |   93.33 | 82-85,88-89
  main.js        |       0 |        0 |       0 |       0 | 1-5
 src/components  |   96.67 |      100 |       0 |   96.67 |
  TaskCard.vue   |     100 |      100 |     100 |     100 |
  TaskForm.vue   |   92.37 |      100 |       0 |   92.37 | 109-117
  TaskList.vue   |     100 |      100 |     100 |     100 |
 src/composables |   94.84 |    88.88 |     100 |   94.84 |
  useTasks.js    |   94.84 |    88.88 |     100 |   94.84 | 11-13,21-22
-----------------|---------|----------|---------|---------|-------------------

 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

Крок 4: Наскрізне тестування (E2E Testing)

Написання мінімум 1-2 сценаріїв для "критичного шляху" користувача (наприклад: "Успішна реєстрація" або "Додавання товару в кошик").

Перевірка коректності відображення елементів інтерфейсу після взаємодії.

```
krostyslav@home:~/users/ros/dev/UniDone$ npm run test:e2e

> unidone@0.0.2 test:e2e
> playwright test


Running 4 tests using 4 workers
  4 passed (1.8s)

To open last HTML report run:

  npx playwright show-report

krostyslav@home:~/users/ros/dev/UniDone$ npx playwright show-report

  Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.
```



Крок 5: Організація "зламаного" тесту (Обов'язково)

Навмисне внесення багу в код.

Перевірка того, що автоматизована система тестування виявила помилку та видала коректний звіт.

```
krostyslav@home:~/users/ros/dev/UniDone$ npm run test:unit

> unidone@0.0.2 test:unit
> vitest


 DEV  v1.6.1 /home/krostyslav/users/ros/dev/UniDone

 ❯ src/composables/__tests__/useTasks.test.js (13)
   ❯ getDaysUntilDeadline (5)
     × коректно розраховує дні до дедлайну (дедлайн через 2 дні)
     ✓ повертає 0, якщо дедлайн сьогодні
     ✓ повертає негативне значення, якщо дата вже минула
     ✓ коректно обробляє дедлайн через тиждень
     ✓ коректно обробляє дедлайн через місяць
   ✓ useTasks (8)
 ✓ src/App.test.js (3)

──────────────────────────────────── Failed Tests 1 ────────────────────────────────────

 FAIL  src/composables/__tests__/useTasks.test.js > getDaysUntilDeadline > коректно розраховує дні до дедлайну (дедлайн через 2 дні)
AssertionError: expected 1 to be 2 // Object.is equality

- Expected
+ Received

- 2
+ 1

 ❯ src/composables/__tests__/useTasks.test.js:24:20
     22|     const result = getDaysUntilDeadline(deadline)
     23|
     24|     expect(result).toBe(2)
       |                    ^
     25|   })
     26|

──────────────────────────────────────────────────────────────────────────────────────[1/1]

 Test Files  1 failed | 1 passed (2)
      Tests  1 failed | 15 passed (16)
   Start at  19:43:56
   Duration  769ms (transform 110ms, setup 15ms, collect 223ms, tests 62ms, environment 579ms, prepare 235ms)


 FAIL  Tests failed. Watching for file changes...
       press h to show help, press q to quit

                                   Ctrl+K to generate command
```

3. Вимоги до звіту

1. Скриншот результатів запуску тестів у консолі (всі тести зелені).
2. Скриншот HTML-звіту про Code Coverage з показниками покриття.
3. Відео або скриншоти проходження E2E тесту в браузері.
4. Короткий аналіз: які типи багів неможливо виявити написаними Unit-тестами.

4. Контрольні запитання

1. У чому різниця між станом (State) та поведінкою (Behavior) під час тестування компонентів?
2. Чому E2E тести вважаються найдорожчими в обслуговуванні, попри їхню високу наочність?
3. Як концепція TDD (Test Driven Development) змінює процес планування роботи менеджером проєкту?

4. Що таке регресійне тестування і як автоматизація допомагає зменшити ризики під час частих релізів?